Quantifying the Conflict Between Parametric and Contextual Memory: A Study of Degradation Metrics on the Qwen3.5 Model Family

ABSTRACT

Quantifying the Conflict Between Parametric and Contextual Memory: A Study of Degradation Metrics on the Qwen3.5 Model Family

RAG is a technique that retrieves documents from a database to inject them in the model's context. LLMs increasingly rely on RAG to use recent or specific information. This creates a conflict between the *parametric memory* (knowledge stored in the model's weights) and the *contextual memory* (knowledge injected in the prompt through RAG). When these two sources of information clash, it creates a conflict: how will then the model choose which information to prefer? We study here this conflict through API updates. We simulate a situation where an LLM trained on a previous version of `NumPy` receives in its context the documentation of the new version. We quantify in this article the tendency of qwen3.5 models to choose their contextual memory over their parametric memory.

1 Introduction

LLMs have demonstrated remarkable capabilities, ranging from natural language understanding to complex code generation. However, these models suffer from limitations inherent to their design: they are prone to hallucinations and remain strictly constrained by the static knowledge in their training corpus. They are thus inherently incapable of reliably answering queries about recent or private information.

To mitigate this issue, the *Retrieval-Augmented Generation* (RAG) technique has established itself as a standard (Lewis *et al.*, 2020). RAG consists of retrieving relevant documents from an external database to inject them directly into the model's context window prior to the generation step.

This new technology, nevertheless, highlights a fundamental tension at the core of the LLM architecture, torn between two sources of knowledge:

- **Parametric memory**: the knowledge statically encoded in the neural network's weights during its training phase.
- **Contextual memory**: the ephemeral information injected directly into the prompt during inference.

A major conflict emerges when these two memories contradict each other. How does the model choose the source of information to prioritize? Longpre et al. (Longpre *et al.*, 2021) demonstrated that LLMs show a strong preference for their training data, even when faced with substituted facts in the context. Xie et al. (Xie *et al.*, 2023) showed that this susceptibility strongly depends on the context's

coherence. More recently, Tang et al. with the *ClashEval* benchmark (Wu *et al.*, 2024) proposed quantifying this "tug-of-war" on natural language question-answering tasks. They confirmed the results of Xie et al. regarding the importance of context coherence on the LLM's choice to rely on its parametric memory or its contextual memory.

Our study extends this work by exploring this conflict in a concrete domain: software engineering and adaptation to API updates. In a real-world environment, libraries evolve. An LLM code assistant was necessarily trained on an older version. If it receives the documentation of the new version in its context (assuming perfect RAG), the expected behavior is that it *systematically* prioritizes the context. However, LLMs are not trained to blindly trust their context over knowledge massively embedded in their weights, such as the use of the `NumPy` library (Harris *et al.*, 2020).

Our contributions are as follows:

- **Development of a functional evaluation framework** via strict execution of the generated code.
- **Introduction of degradation metrics** allowing the isolation of the loss of syntactic coherence from the loss of intelligence.
- **Quantification of the performance losses** of the qwen3.5 model family on two different perturbations.

2 Methodology and Experimental Framework

Our goal was therefore to measure and evaluate to what extent LLMs are capable of adapting to information that is not only new but in direct conflict with the data they were trained on. To measure this adaptation capacity and, where applicable, the performance loss linked to this need for adaptation, we first generated counterfactual documentations of three types (ultra-minimal, minimal, and explanatory). Next, we provided this documentation to the LLM via the llama.cpp inference engine (Gerganov, 2023) and the high-level ollama wrapper (Ollama Contributors, 2024). We provided the documentation to the LLM directly within its context window, ensuring that the documentation size was smaller than the LLM's usual maximum context window size. We used the DS1000 benchmark (Lai *et al.*, 2023) as the base of problems to solve, keeping only the problems related to the `NumPy` library (totaling 159 problems to solve) and modifying the problem statements to ensure the `NumPy` perturbation was also present. Then, we retrieved the code generated by the LLM via parsing methods. Finally, we performed a functional evaluation of the generated code to ensure two things: First, that the code properly uses the modifications intended to represent a library update. Second, that the code utilizing the appropriate modifications works correctly and returns the right values on the various DS1000 test cases. Figure 1 summarizes our pipeline.

2.1 Counterfactual Documentations and Perturbations

The documentation generation is deterministic. We evaluate three levels of documentation: **Ultra-minimal** (catalog of names), **Minimal** (names, arguments, and signature), and **Explanatory** (textual explanation of the perturbation rule).

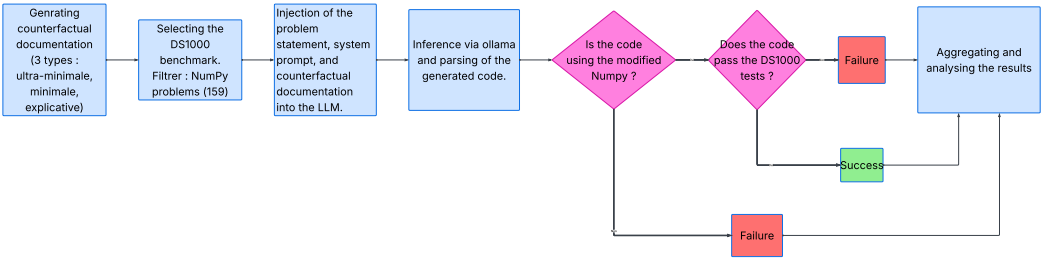

Figure 1: Evaluation pipeline: from the design of the counterfactual documentation to the dual dynamic evaluation via wrappers and AST cleaning.

We apply two types of perturbations applied to the terminal attribute:

- **Suffix _v2:** Systematic addition of `_v2` (e.g., `np.mean_v2`).
- **Capital:** Capitalization of the first letter (e.g., `np.Mean`).

2.2 Dual Functional Evaluation and AST Parsing

Unlike standard textual evaluations, our pipeline validates the code through a *dual dynamic execution*:

1. **Evaluation with injection (Wrapper):** The code is passed to a strict wrapper that intercepts accesses to the `np` object. If the perturbed convention is not respected, a `MissingSuffixError` is raised.
2. **Control evaluation:** The same code, after algorithmic removal of the perturbations via Abstract Syntax Tree (AST) cleaning, is executed with the standard `numpy` import.

Thus, the code is executed on both the perturbed library and the original library. This method allows us to verify whether a code that fails to apply the perturbation was nevertheless algorithmically correct with the classic version of `NumPy`.

3 Results and Analysis: The Qwen3.5 Family

We evaluated the Qwen3.5 family (Bai *et al.*, 2023) (0.8B, 2B, 4B, 9B, 27B, 35B) in local inference via ollama, using the probabilistic $pass@10$ metric (Chen *et al.*, 2021).

3.1 Scaling Law and Baseline Capabilities

On the control folds (without conflict), performance strictly increases with the number of parameters. The success rate goes from 15 % (0.8B) to 81 % (35B). As illustrated in Figure 2, this evolution seems

to follow a logarithmic relationship:

$$y = a \cdot \log(p) + b \tag{1}$$

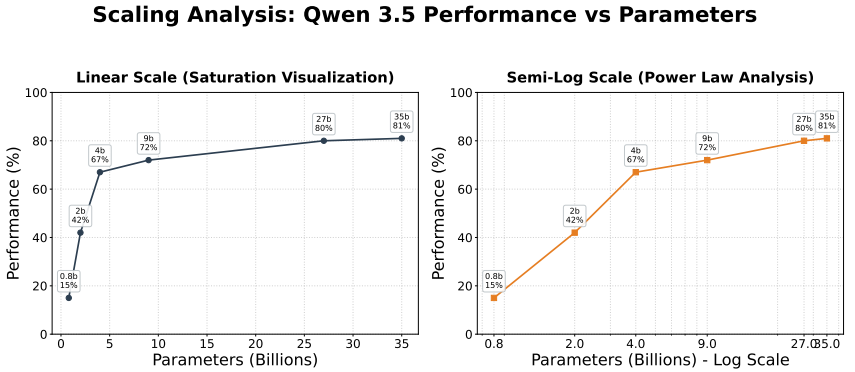

Figure 2: Performance of the Qwen3.5 family models on the control folds as a function of the number of parameters (left) and the logarithm of the number of parameters (right).

3.2 Initial Analysis of Performance on Counterfactual Folds

In this section, we will analyze the evaluation results summarized in Figure 3.

For the fluidity of the analyses we will conduct subsequently, it is important to note that although qwen3.5:35b is from the same family as the other models (same training corpus, same post-training structure), its architecture differs from the others as it is based on a Mixture-of-Experts (MoE) architecture, which accelerates inference but can have a negative impact on performance, particularly on reasoning.

We also observe that LLMs systematically perform better during test folds with explanatory documentation rather than with minimal or ultra-minimal documentations. It is as if having to generalize the perturbation causes a drop in the LLM's competence. Thus, assisting the LLM by explaining the rule to it, rather than letting it infer it from a large number of examples, seems to leave it with fewer opportunities to make mistakes. This is unfortunate in our case, as we would want the LLM to be able to successfully use the new library documentation on its own.

A quick note on the interpretation of Figure 3: It could happen that the sum of the success percentage on the counterfactual library evaluation and the original library evaluation exceeds the success percentage on the control fold. Indeed, since the evaluation is functional and some problems can be solved without using the `NumPy` library, it happens that some solutions are counted as correct for both the base library and the counterfactual library. To avoid this scenario, we decided to exclude functions that pass both evaluations.

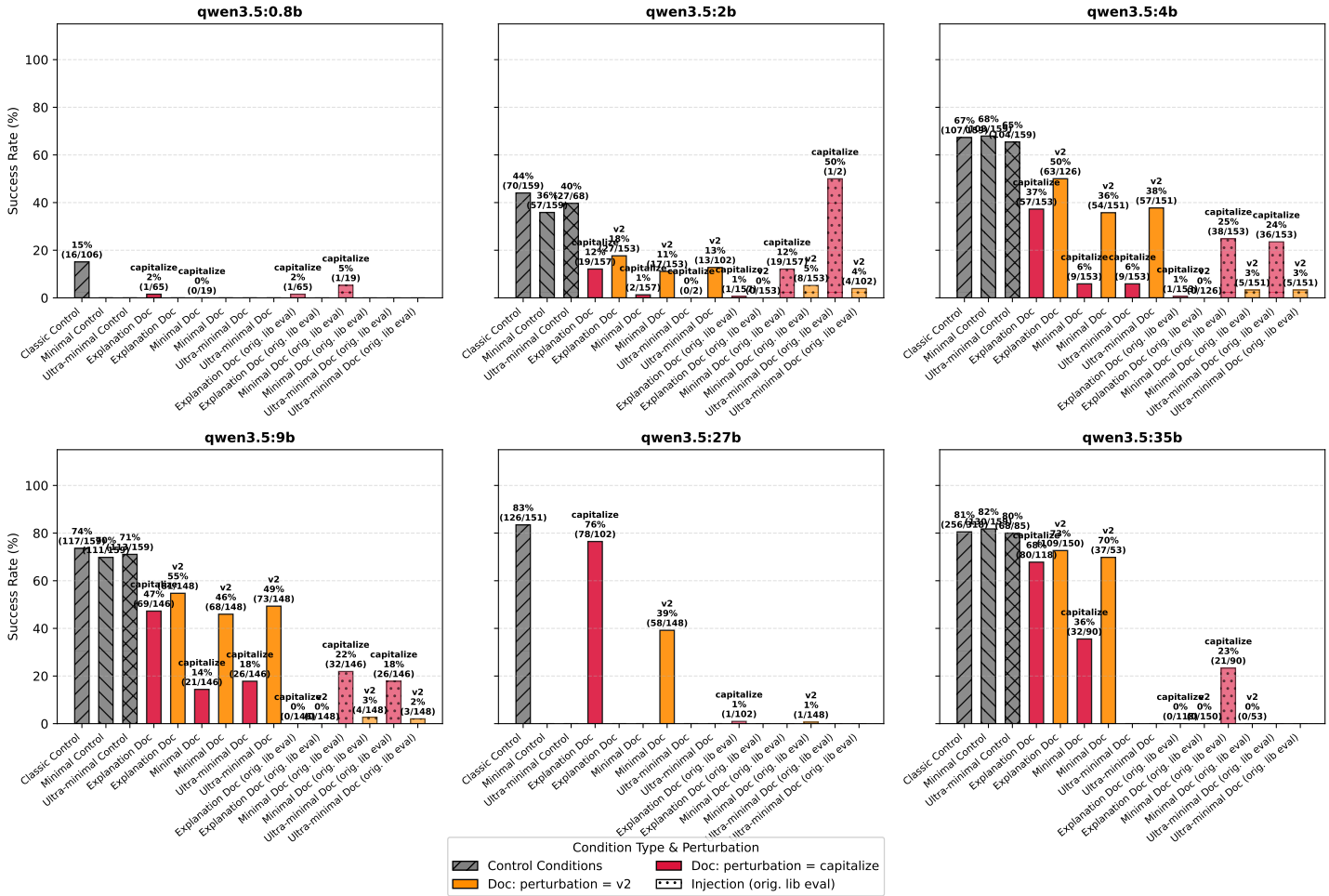

Figure 3: Performances of the qwen3.5 family models on the control folds (classic, minimal, and ultra-minimal documentation) and injection folds with the _v2 and capital perturbations (with explanatory, minimal, and ultra-minimal documentations in context) evaluated on the original library and the counterfactual library.

3.3 Degradation Metrics

We introduce here a few definitions that will later allow us to conduct more in-depth analyses: Let $\mathcal{E}_{ctrl}$ and $\mathcal{E}_{inj}$ be the sets of control and injection evaluations. We define the success subsets:

$$\begin{aligned} S_{ctrl} &= \{e \in \mathcal{E}_{ctrl} \mid \text{control_passed}(e)\} \\ S_{inj} &= \{e \in \mathcal{E}_{inj} \mid \text{injection_passed}(e)\} \\ S_{inj_ctrl} &= \{e \in \mathcal{E}_{inj} \mid \text{control_passed}(e)\} \end{aligned}$$

where control_passed means that the generated code works when executed with the original library, and injection_passed when it works with the wrapper. Note that S_{ctrl_inj} is not considered since we assume this set is inherently empty (why would the LLM generate perturbed code when the perturbation was not even specified to it?).

1. Performance loss L_{perf} measures the strict drop in success on the task we ultimately care about:

$$L_{perf} = \frac{|S_{ctrl}|}{|\mathcal{E}_{ctrl}|} - \frac{|S_{inj}|}{|\mathcal{E}_{inj}|} \tag{2}$$

2. Competence loss L_{comp} penalizes the inability to produce valid code for at least one of the two libraries:

$$L_{comp} = \frac{|S_{ctrl}|}{|\mathcal{E}_{ctrl}|} - \frac{|S_{inj} \cup S_{inj_ctrl}|}{|\mathcal{E}_{inj}|} \tag{3}$$

3. Intelligence loss L_{int} relies on the set C of confusions (assertion errors):

$$C = \{e \in \mathcal{E}_{inj} \mid e \notin (S_{inj} \cup S_{inj_ctrl}) \wedge (\text{stdout}(e) \in \text{Err}_{assert} \vee \text{stdout_control}(e) \in \text{Err}_{assert})\} \tag{4}$$

$$L_{int} = \frac{|C|}{|\mathcal{E}_{inj}|} \tag{5}$$

4. Coherence loss L_{coh} is defined as the residual of the competence loss:

$$L_{coh} = L_{comp} - L_{int} \tag{6}$$

3.4 Return to the Analysis

We systematically observe (albeit to varying degrees depending on the model size, the perturbation type, and the documentation type) a performance loss in the sense that the LLM, which previously succeeded in solving a problem on the control fold, produces a solution that executes neither with the counterfactual library nor with the original library.

Nevertheless, we can see that on the explanatory folds, the model loses less *competence* than on the minimal and ultra-minimal folds. Indeed, the sum of the evaluations on the counterfactual library and the original library gives, give or take, the success percentage on the control fold (81% success on the control fold for qwen3.5:35b, 68% on the capital perturbation evaluated on the counterfactual library, and 0% evaluated on the original library, resulting in a performance loss of 12%, see Figure 3). This

phenomenon strengthens as the models get larger. The more parameters the model has, the smaller the *competence* loss is on the explanatory documentation relative to the baseline success rate.

On the explanatory documentations, the models increase their *performance* (and their *competence*) first by improving their *coherence* (from qwen3.5:0.8b to qwen3.5:9b), then by improving their *intelligence* (qwen3.5:27b and qwen3.5:35b), as seen in the following figures.

However, we do not observe the same behavior on the test folds performed with minimal documentations. Indeed, in this case, we rather observe a linear improvement in *intelligence* as a function of the logarithm of the number of parameters, while the loss of *coherence* remains relatively stable as a function of the number of parameters. We thus observe a mutation in the LLMs' improvement regime based on the number of parameters and depending on the type of documentation used: explanatory or resembling actual documentation. In the explanatory case, the increase in the number of parameters mostly leads to an initial gain in coherence, followed by a gain in intelligence. In the minimal and ultra-minimal cases, intelligence is constantly gained, but coherence struggles to keep up.

Let us note something surprising on this graph: The qwen3.5:0.8b model has a negative coherence loss. This can be explained by the fact that the intelligence loss is greater than the competence loss, because where the LLM previously failed on the control fold by making syntax or variable definition errors, it now fails by producing code that executes correctly but contains bugs. This behavior might be more expected on models with weak reasoning capabilities, as is the case here with qwen3.5:0.8b. We can explain this phenomenon in another way: As we did not have time to run enough tests on the minimal capitalize fold for qwen3.5:0.8b, we were only able to evaluate it on 19 problems, whereas it was evaluated on 106 problems during the control fold. If the problems on which the model fails due to intelligence are more concentrated among the first problems, this biases the results towards an intelligence loss rather than a coherence loss.

We can also note the slight loss of intelligence and coherence, and therefore of relative competence and performance, of qwen3.5:35b compared to qwen3.5:27b, even though the latter was worse on the control folds. This may be explained by the MoE architecture of qwen3.5:35b compared to the dense architecture of qwen3.5:27b.

Finally, it seems that the models of the qwen3.5 family have an easier time with the _v2 perturbation than with the capital perturbation. This is perhaps due to the fact that the _v2 perturbation is easier (one simply needs to add the same sequence of tokens to the end of the function call, regardless of the function, whereas one must replace a letter with its capital equivalent in the case of the capital perturbation). This might also be because the _v2 perturbation seems more plausible than the capital perturbation, or a mix of both.

4 Study Limitations

To guarantee strict control of the experimental variables, all our counterfactual documentation was generated algorithmically (via regular expressions and parsing) from existing *docstrings*. Although this method ensures perfect reproducibility, it produces highly rigid text. In reality, developers face release notes, migration guides, or annotated code examples. A complementary avenue would be to use an instructed LLM to rewrite and generate more natural and varied counterfactual documentations, closer to what a RAG system would retrieve in practice. Our systematic modifications inevitably present an artificial character. Although an alteration specific to each method would better reflect the

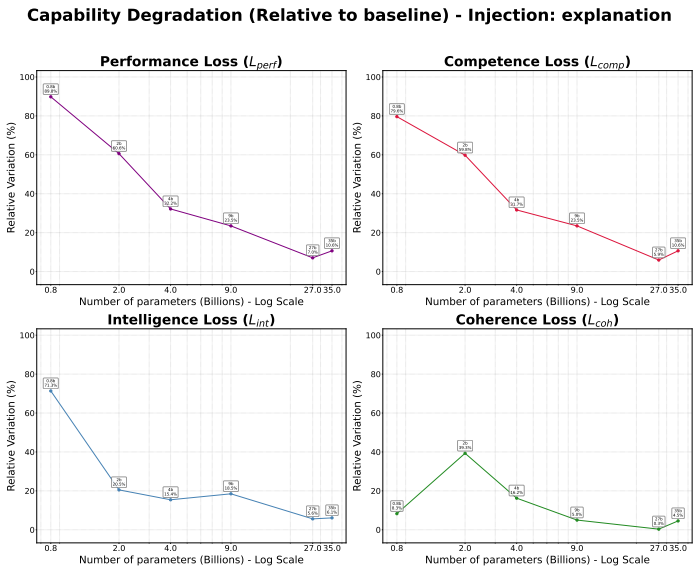

Figure 4: Degradation metrics relative to the success rate (control) for **explanatory** documentations (Qwen3.5 family).

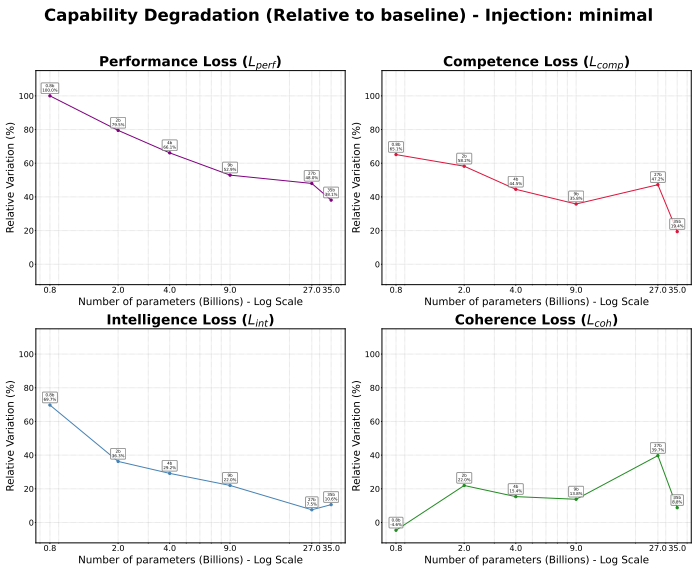

Figure 5: Degradation metrics relative to the success rate (control) for **minimal** documentations (Qwen3.5 family).

reality of a software update, we favored a systematic approach for automation reasons. The direct consequence of this lack of plausibility is the LLM's increased propensity to ignore the context and fall back on its parametric memory. Furthermore, our goal is not to test whether the model can apply a uniform syntactic transformation to an entire API—an exercise that would undoubtedly be easier for it but disconnected from real-world development conditions.

Similar to the limitations highlighted by Wu et al. in *ClashEval* (Wu *et al.*, 2024), whose questions relied on direct facts without requiring multi-step logic, our evaluation on the NumPy subset of DS-1000 focuses on short and isolated code snippets. The integration of this conflicting information in more complex software engineering tasks (synthesis of multiple documents, modifications across an entire repository) remains an open question.

5 Potential Solutions to the Initial Problem

To solve the initial problem of LLM adaptation to an API update, we propose, based on our results, the following potential avenues:

- Perform code generation using two instances of the model. The first instance would analyze the updated documentation and produce explanatory documentation for the second LLM instance, which could then generate code based on this explanatory documentation.
- Fine-tune the model to be much more receptive to the context. This solution must be considered with caution, as if adversarial context were to find its way into the LLM's context, it would almost certainly produce corrupted code.

6 Conclusion

This work aimed to rigorously quantify the ability of large language models to arbitrate between their parametric memory and a counterfactual contextual memory within a deterministic and functional evaluation framework: code generation under API update constraints. By transposing the *ClashEval* (Wu *et al.*, 2024) paradigm—previously confined to natural language question-answering—to the software engineering domain, we proposed an original evaluation pipeline based on a dual dynamic execution (counterfactual library via *wrapper* and original library) and syntactic cleaning via AST parsing, ensuring that each verdict is functional rather than textual.

6.1 Key Results.

Our experiments yield several conclusions:

First, no tested model manages to fully switch to the contextual memory: even the best models (like Qwen3.5:35b) show a performance drop of around 12 points on the capital perturbation when the rule is explicitly stated, and of 44 points when it must be inferred from implicit documentation. The resistance of the parametric memory is a robust phenomenon, even if it diminishes—without entirely disappearing—with model size.

Second, the introduction of our degradation metrics (L_{perf}, L_{comp}, L_{int}, L_{coh}) reveals that the model's adaptation takes two distinct trajectories depending on the nature of the provided documentation. With *explanatory* documentation, the overall loss of competence is minimized. Increasing the model's parameter count operates a sequential correction: it first stabilizes the assimilation of the new convention (gain in coherence), then restores the algorithmic resolution capacity (gain in intelligence). Conversely, faced with *implicit* documentation (minimal or ultra-minimal), while intelligence grows logarithmically with model size, coherence stagnates against the contextual load. The model fails to maintain the counterfactual convention, illustrating the limits of a RAG system providing non-explanatory contexts.

Third, in the so-called control evaluation mode (without perturbation), we do not notice major performance differences whether we provide no documentation, minimal documentation, or ultra-minimal documentation. We initially thought that context size would tend to weaken the model's performance. We observe similar results in the injection mode, where minimal and ultra-minimal documentations offer similar performances.

Scope and originality of the contribution. Beyond the numbers, another of our contributions is methodological. By replacing the semantic proximity measure used in *ClashEval* with strict functional validation by execution, we propose a more robust evaluation protocol. This *dual dynamic execution*, made possible by our system of *wrappers* and AST cleaning, constitutes a reusable methodology for any benchmark wishing to evaluate the parametric memory/contextual memory trade-off in a code generation context. The framework thus defined is generic: by writing new wrappers, it is directly extensible to other libraries (Pandas, PyTorch, Scikit-learn), to other types of perturbations (semantic, behavioral), or to other model families.

References

BAI J., BAI S., CHU Y. & ET AL. (2023). Qwen technical report. *arXiv preprint arXiv:2309.16609*.

CHEN M., TWOREK J., JUN H., YUAN Q., PINTO H. P. D. O., KAPLAN J. *et al.* (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

GERGANOV G. (2023). llama.cpp. https://github.com/ggml-org/llama.cpp.

HARRIS C. R., MILLMAN K. J., VAN DER WALT S. J., GOMMERS R., VIRTANEN P., COURNAPEAU D., WIESER E., TAYLOR J., BERG S., SMITH N. J., KERN R., PICUS M., HOYER S., VAN KERKWIJK M. H., BRETT M., HALDANE A., DEL RÍO J. F., WIEBE M., PETERSON P., GÉRARD-MARCHANT P., SHEPPARD K., REDDY T., WECKESSER W., ABBASI H., GOHLKE C. & OLIPHANT T. E. (2020). Array programming with NumPy. *Nature*, **585**(7825), 357–362. DOI : 10.1038/s41586-020-2649-2.

LAI Y., LI C., WANG Y., ZHANG T., ZHONG R., ZETTLEMOYER L., YIH S. W.-T., FRIED D., WANG S. & YU T. (2023). Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning (ICML)*, p. 18319–18345: PMLR.

LEWIS P., PEREZ E., PIKTUS A., PETRONI F., KARPUKHIN V., GOYAL N., KÜTTLER H., LEWIS M., YIH W.-T., ROCKTÄSCHEL T. *et al.* (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, **33**, 9459–9474.

LONGPRE S., PERISETLA K., CHEN A., RAMESH N., DUBOIS C. & SINGH S. (2021). Entity-based knowledge conflicts in question answering. *arXiv preprint arXiv:2109.05052*.

OLLAMA CONTRIBUTORS (2024). Ollama. https://github.com/ollama/ollama.

WU K., WU E. & ZOU J. (2024). Clasheval: Quantifying the tug-of-war between an llm's internal prior and external evidence. *arXiv preprint arXiv:2404.10198*.

XIE J., ZHANG K., CHEN J., LOU R. & SU Y. (2023). Adaptive chameleon or stubborn sloth: Unraveling the behavior of large language models in knowledge conflicts. *arXiv preprint arXiv:2305.13300*.